UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
Facultad de Ingeniería en Sistemas Computacionales
Departamento de Programación de Computadoras

PROYECTO PARCIAL 2

Book&Go Lite

Gestor de citas con reseñas

Negocio elegido: **Navaja & Estilo** (barbería)

Asignatura: Desarrollo Web
Profesor: Giovani Sánchez

Grupo 4

Josua Martinez
Cristhofer Villarreal
Brayan Mela
Ian Ramos

Fecha de entrega: 12 de noviembre de 2026
Repositorio: https://github.com/DSCrip07/PW2026_BookGoLite

1. Integrantes y responsabilidades

El proyecto se desarrolló en equipo de cuatro estudiantes, con un módulo asignado a cada uno según la distribución sugerida en el enunciado.

Integrante	Usuario de GitHub	Módulo asignado
Josua Martinez	DSCrip07	Autenticación y sesiones
Cristhofer Villarreal	cristhofer-11	Catálogo y administración
Brayan Mela	brayanmela17-hub	Horarios y reservas
Ian Ramos	ianramos21-star	Reseñas, interfaz y pruebas de seguridad

2. Negocio elegido

Navaja & Estilo es una barbería que gestiona sus citas por internet. Los clientes se registran, exploran los servicios, reservan un horario con cupos disponibles, pueden cancelar su reserva y, una vez realizada la atención, publicar una reseña. El administrador gestiona categorías, servicios, horarios y el estado de las reservas.

Categorías: Cortes, Barba y Tratamientos. **Servicios de ejemplo:**

Servicio	Categoría	Precio	Duración
Corte clásico	Cortes	$8.00	30 min
Corte degradado (fade)	Cortes	$10.00	45 min
Corte infantil	Cortes	$6.00	30 min
Perfilado de barba	Barba	$6.00	30 min
Afeitado con toalla caliente	Barba	$9.00	40 min
Tratamiento capilar	Tratamientos	$12.00	60 min

Tecnologías: PHP 8.2 (PDO), MySQL/MariaDB, HTML5, CSS3 y JavaScript, sobre XAMPP (Apache). **Plantilla o biblioteca visual:** ninguna; el diseño (paleta carbón y dorado, tipografía y componentes) es CSS propio, sin Bootstrap ni otras bibliotecas externas.

3. Diagrama entidad-relación

El esquema tiene siete tablas relacionadas, todas con clave primaria: **roles, usuarios, categorias, servicios, horarios, reservas y resenas**.

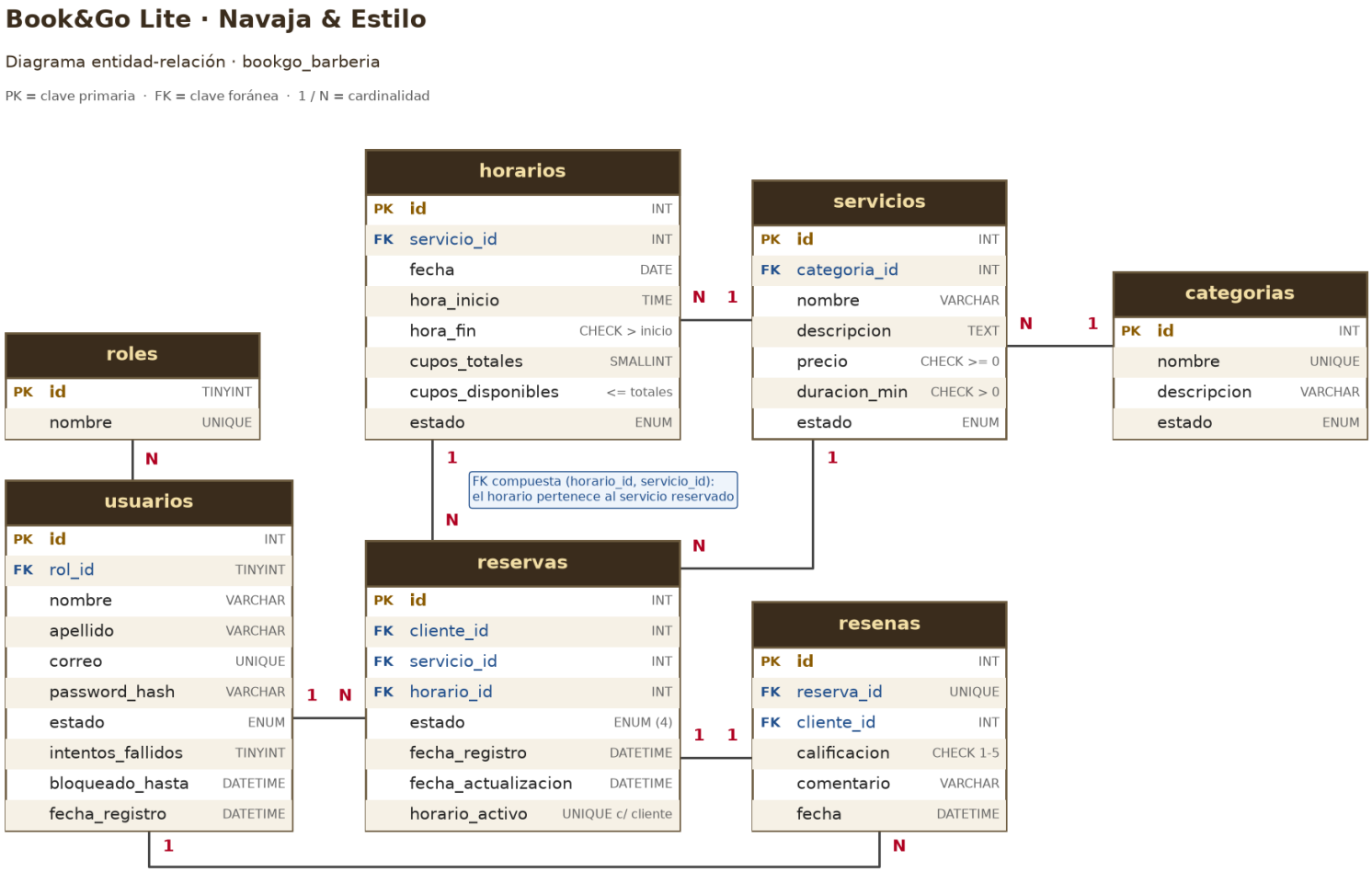


Figura: diagrama entidad-relación de bookgo_barberia

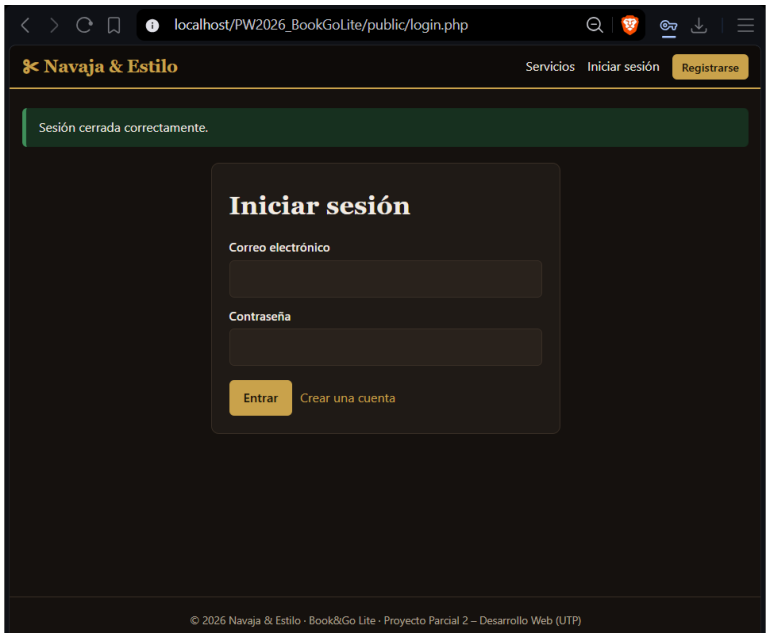
Cómo las relaciones impiden datos inconsistentes

- **Horario del servicio correcto:** la reserva tiene una clave foránea compuesta (horario_id, servicio_id) hacia horarios; la base de datos rechaza reservar un horario que no pertenece al servicio indicado.
- **Sin doble reserva activa:** una columna generada (horario_activo) solo toma valor mientras la reserva está PENDIENTE o CONFIRMADA; con un índice único (cliente_id, horario_activo) un cliente no puede tener dos reservas activas en el mismo horario.
- **Cupos válidos:** cupos_disponibles es sin signo (nunca negativo) y una restricción CHECK impide que supere los cupos totales.
- **Datos válidos:** CHECK para precio mayor o igual a 0, duración mayor que 0, hora final posterior a la inicial y calificación entre 1 y 5.
- **Unicidad:** el correo del usuario es único y reserva_id es único en resenas (una reseña por reserva).
- **Integridad referencial:** todas las relaciones usan claves foráneas; los registros con dependencias no se eliminan físicamente, se desactivan.

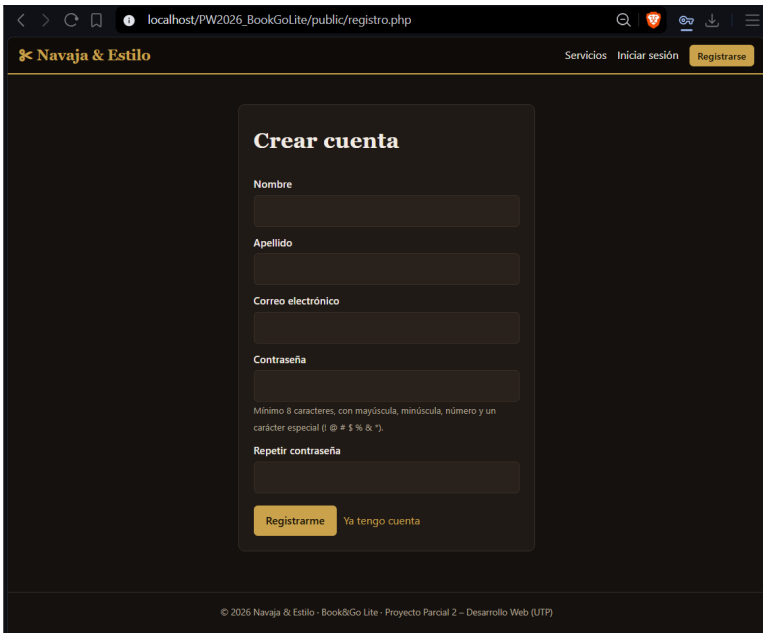
4. Módulos y capturas de pantalla

4.1 Autenticación y sesiones

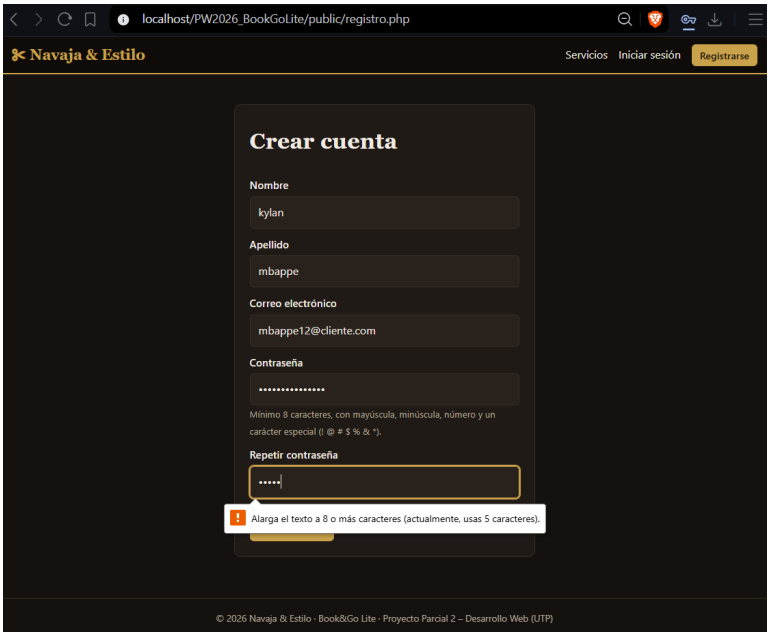
Registro público (crea únicamente usuarios CLIENTE), inicio y cierre de sesión, validación de credenciales y estado, protección CSRF, política de contraseñas (8 o más caracteres con mayúscula, minúscula, número y carácter especial) y bloqueo temporal tras cinco intentos fallidos. El ADMIN se crea con un script de consola.



Inicio de sesión (mensaje tras cerrar sesión)



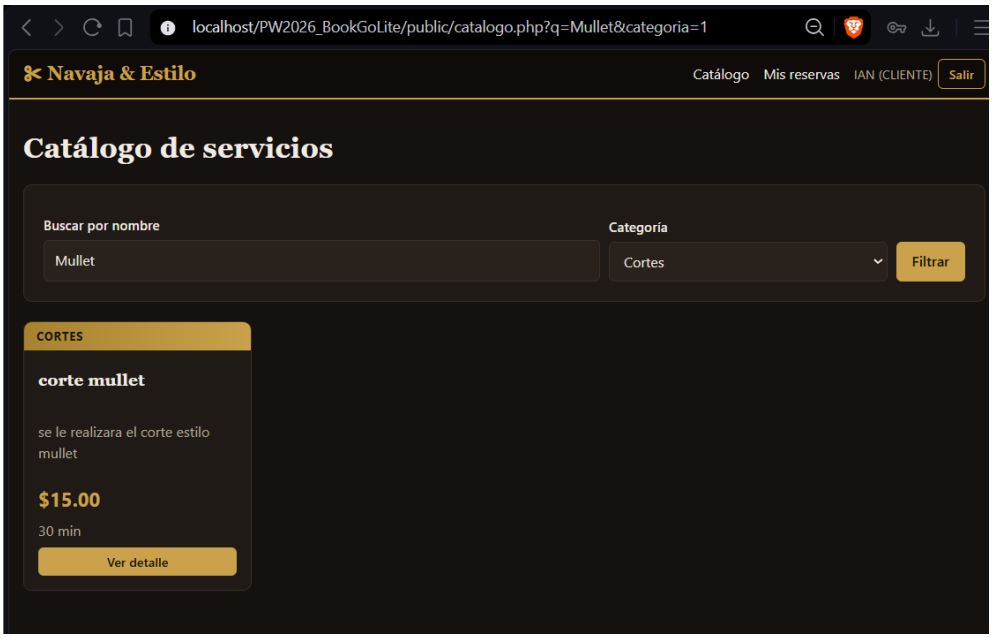
Formulario de registro con la regla de contraseña



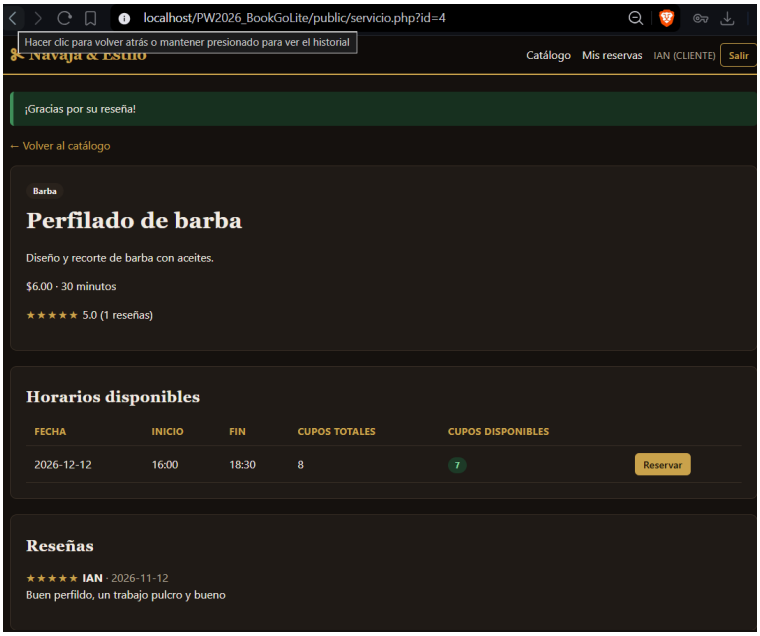
Validación de la contraseña en el formulario

4.2 Catálogo y administración

Catálogo con búsqueda por nombre y filtro por categoría, detalle del servicio y panel del ADMIN con CRUD de categorías y servicios (crear, editar, desactivar y eliminar solo si no tiene dependencias).



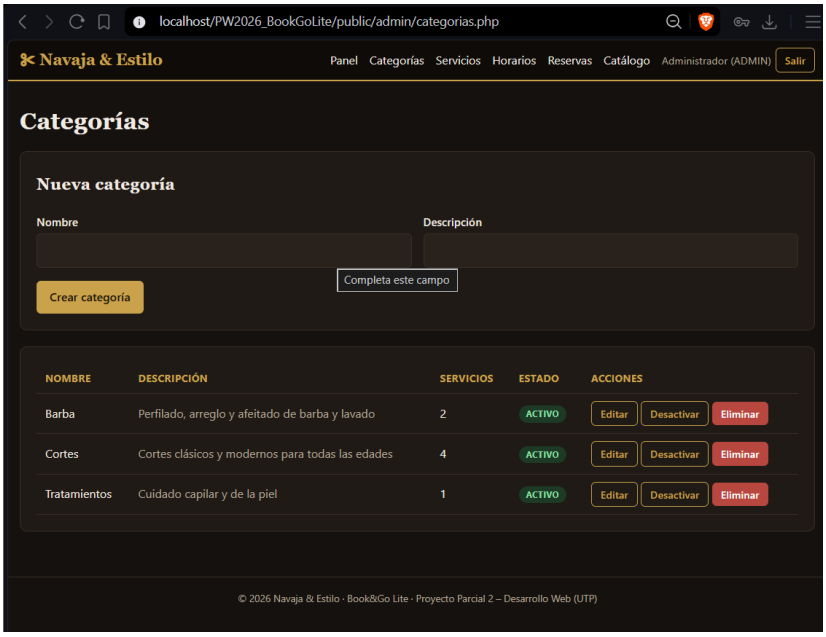
Catálogo con búsqueda y filtro por categoría



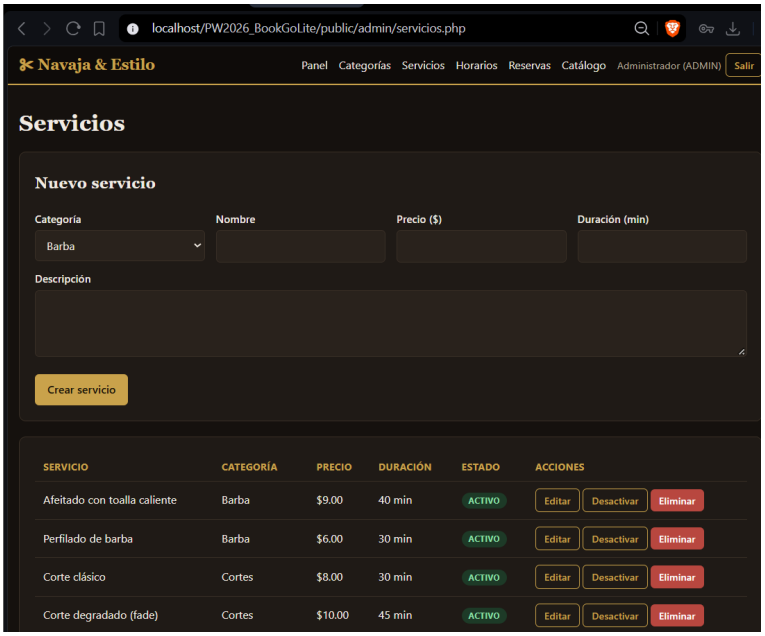
Detalle del servicio: horarios con cupos y reseñas



Panel de administración



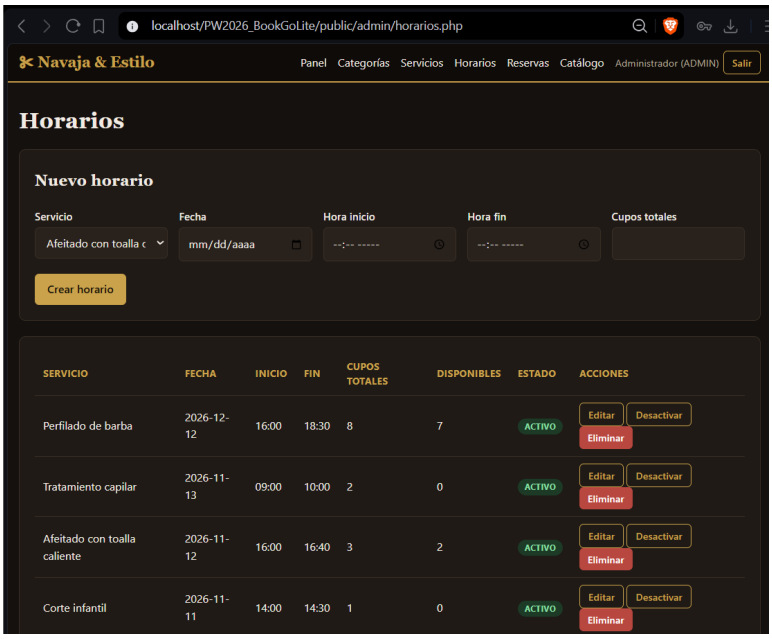
Mantenimiento de categorías



Mantenimiento de servicios

4.3 Horarios y reservas

CRUD de horarios con validaciones (no en el pasado, hora final posterior a la inicial), reservas con control de cupos en transacción, cancelaciones y cambio de estados por el ADMIN (PENDIENTE, CONFIRMADA, REALIZADA, CANCELADA).



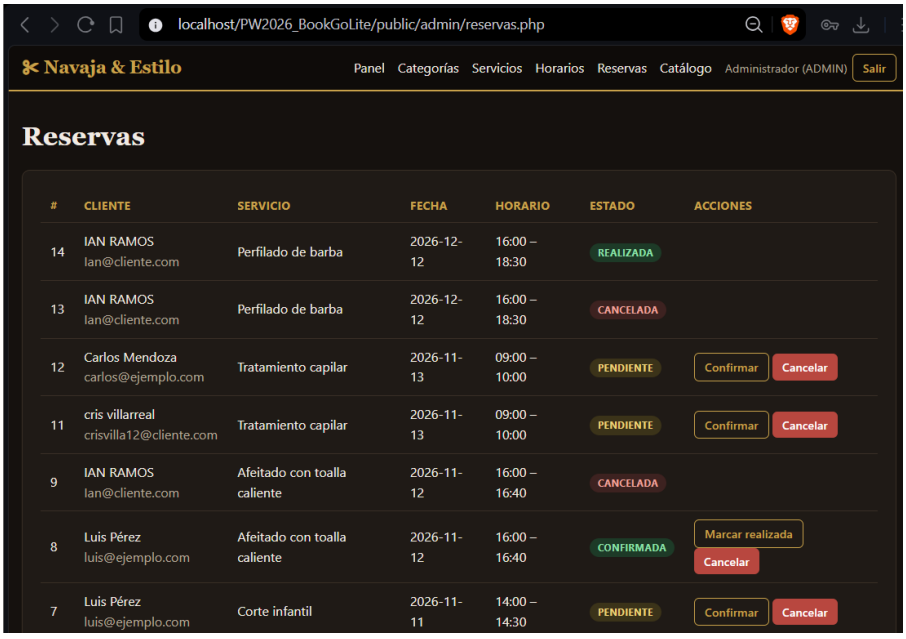
Mantenimiento de horarios con cupos totales y disponibles



Reserva creada en estado PENDIENTE



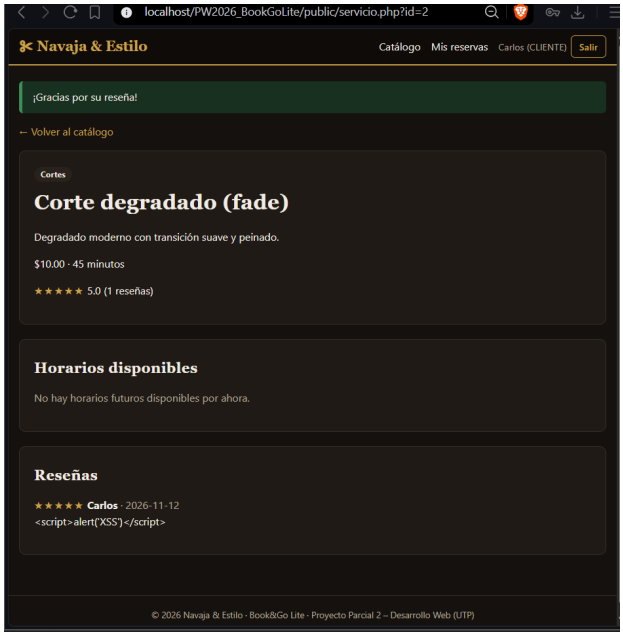
Reserva cancelada: se devuelve el cupo



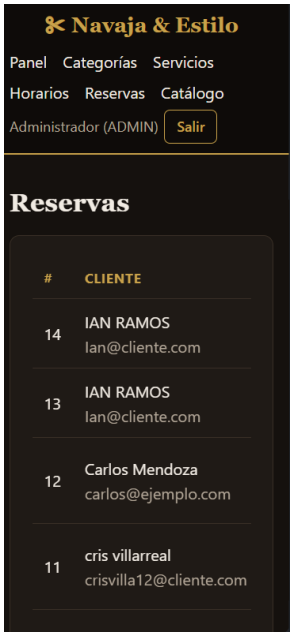
Gestión de reservas por el ADMIN

4.4 Reseñas, interfaz y diseño adaptable

Un cliente puede publicar una sola reseña (1 a 5 y comentario) por cada reserva propia REALIZADA; las reseñas se muestran en el detalle del servicio. La interfaz se adapta a computadora y teléfono.



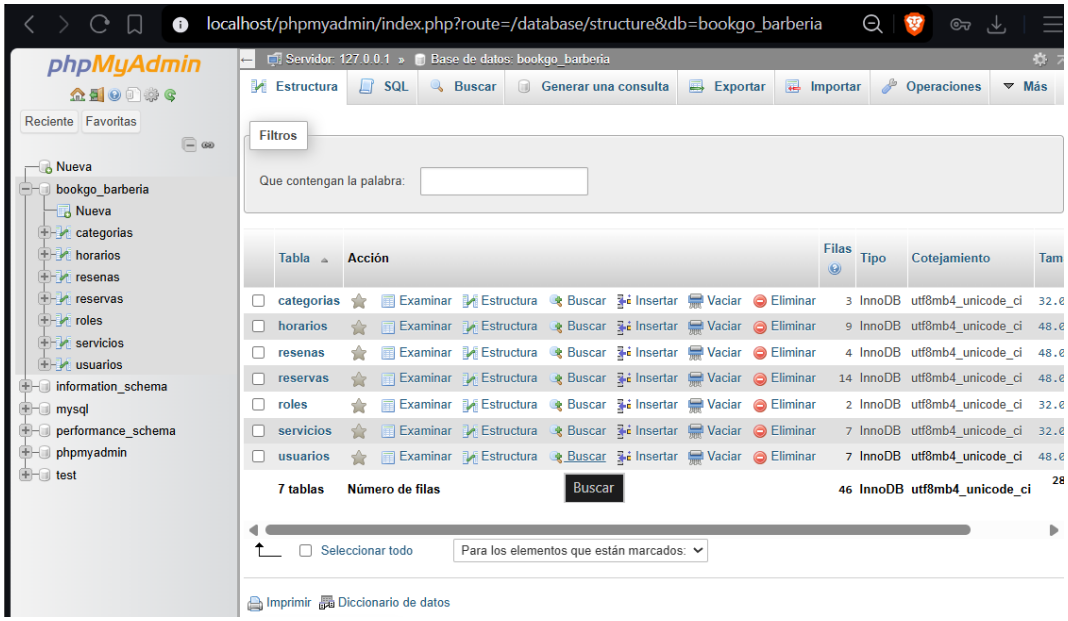
Reseña publicada y mostrada en el detalle del servicio



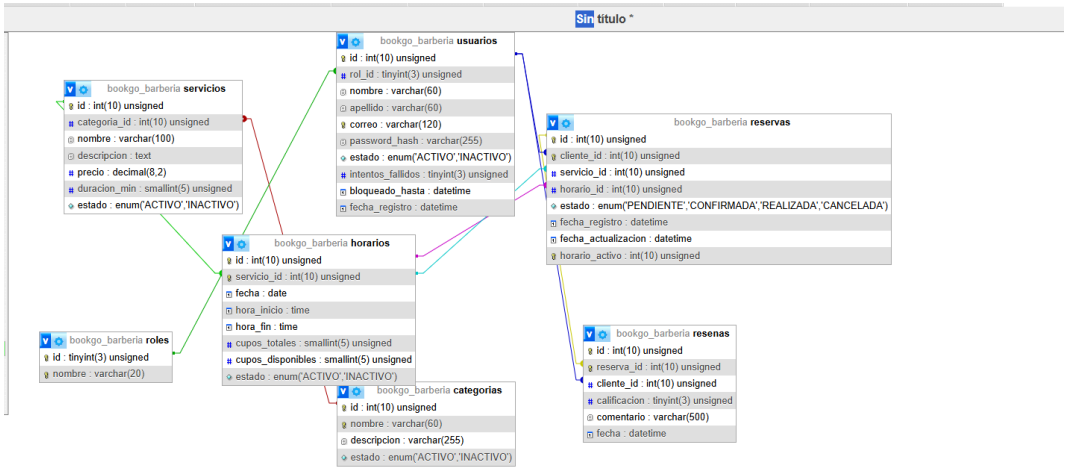
Vista en pantalla de teléfono

4.5 Base de datos

Esquema de siete tablas importable desde database/schema.sql, con datos de prueba.



Las siete tablas en phpMyAdmin



Relaciones entre tablas (vista Diseñador)

5. Reglas de negocio de las reservas

- El cliente elige un servicio y un horario futuro con cupos disponibles. Al reservar, la reserva queda **PENDIENTE** y se descuenta un cupo.
- El cupo se descuenta y la reserva se registra en **una sola transacción** (SELECT ... FOR UPDATE y UPDATE ... WHERE cupos_disponibles > 0), de modo que dos clientes simultáneos no pueden superar el límite. No depende de deshabilitar un botón en HTML.
- Un mismo cliente no puede mantener dos reservas activas para el mismo horario.
- El ADMIN pasa PENDIENTE a CONFIRMADA y luego a REALIZADA (no puede saltarse pasos) y puede cancelar.
- El cliente solo cancela reservas propias que no estén REALIZADAS y cuya cita no haya comenzado.
- Al cancelar una reserva PENDIENTE o CONFIRMADA se devuelve exactamente un cupo; cancelar de nuevo no devuelve cupos adicionales. Una reserva REALIZADA no se cancela ni devuelve cupos.
- Los cupos nunca son negativos ni superiores al total.

6. Controles de seguridad y pruebas

Todas las pruebas se realizaron únicamente en la aplicación de desarrollo (localhost). El detalle de cada una está en **security/SECURITY.md** y las capturas en **security/evidencias/**.

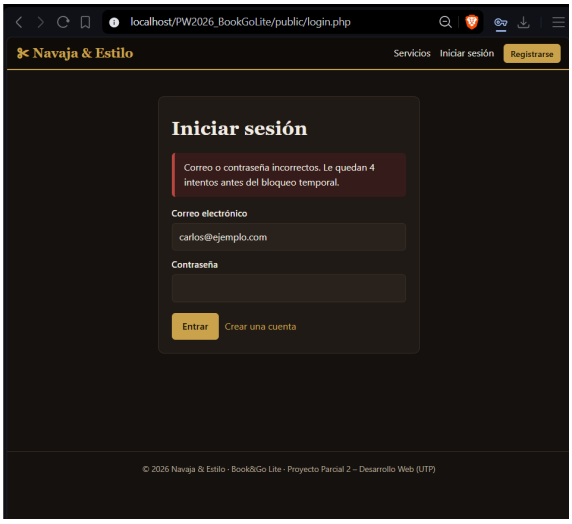
Control	Implementación
Contraseñas	password_hash() / password_verify(); mínimo 8 caracteres con mayúscula, minúscula, número y carácter especial
Sesiones	session_regenerate_id al iniciar sesión; sesión y cookie destruidas al salir; cookie HttpOnly y SameSite=Lax (Secure con HTTPS)
Intentos de login	Bloqueo temporal de 5 minutos tras 5 intentos fallidos, sin revelar si el correo existe
SQL Injection	PDO y consultas preparadas con parámetros
XSS	htmlspecialchars con ENT_QUOTES y UTF-8; validación de entradas
CSRF	Token por sesión verificado en cada operación que cambia datos
Autorización	Rol y propiedad de cada recurso verificados en el servidor
Cabeceras HTTP	Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy
Errores	Mensajes genéricos al usuario; el detalle solo se registra en el servidor

Prueba 1 – SQL Injection (búsqueda y login)

Objetivo: Verificar que una entrada maliciosa no altera la consulta ni evita la autenticación.
Resultado observado: La búsqueda con ' *OR '1'='1* devuelve 0 servicios (no todos) y el login con ese texto como contraseña no inicia sesión.
Control aplicado: PDO con consultas preparadas y parámetros enlazados en toda la aplicación.



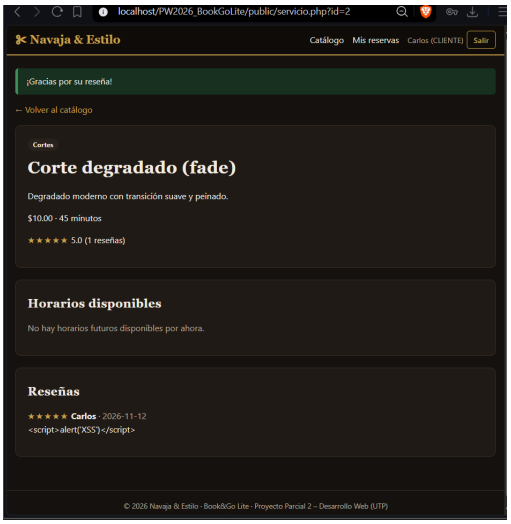
Búsqueda con intento de inyección SQL



Login con intento de inyección SQL

Prueba 2 – XSS en comentario de reseña

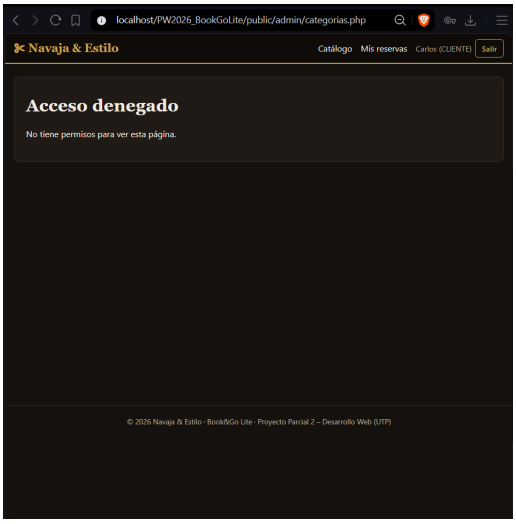
Objetivo: Verificar que un script ingresado se muestra como texto y no se ejecuta.
Resultado observado: El comentario <script>alert('XSS')</script> se muestra literalmente y no aparece ninguna alerta.
Control aplicado: Salida codificada con htmlspecialchars (ENT_QUOTES, UTF-8), validación de entrada y Content-Security-Policy.



Script mostrado como texto en la reseña

Prueba 3 – Acceso directo de un CLIENTE a una página ADMIN

Objetivo: Comprobar que el rol se valida en PHP y no solo en el menú.
Resultado observado: Un cliente que escribe la URL de una página de administración recibe «Acceso denegado» (HTTP 403); sin sesión se redirige al login.
Control aplicado: Función requiere_rol('ADMIN') al inicio de cada página de administración.



Acceso denegado a un cliente

Prueba 4 – Acceso a la reserva de otro cliente

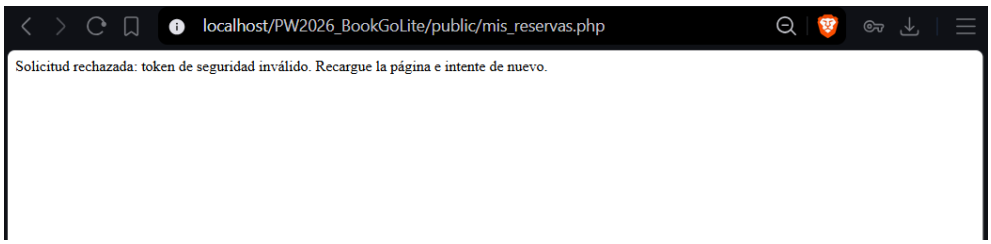
Objetivo: Comprobar que cada cliente solo ve o modifica lo que le pertenece.
Resultado observado: Al abrir la reserva de otro cliente se muestra «Solo puede reseñar sus propias reservas»; un intento de cancelar una reserva ajena responde «Reserva no encontrada» y no cambia el estado.
Control aplicado: Consultas filtradas siempre por el id del usuario de la sesión.



Intento de acceder a una reserva ajena

Prueba 5 – Formulario sin token CSRF válido

Objetivo: Verificar que una operación sin token (o con token falso) es rechazada.
Resultado observado: Aparece «Solicitud rechazada: token de seguridad inválido» (HTTP 403) y la reserva no cambia de estado.
Control aplicado: Token aleatorio por sesión, verificado con hash_equals en cada POST que modifica datos.



Operación rechazada por token CSRF inválido

Prueba 6 – Cinco intentos de login fallidos

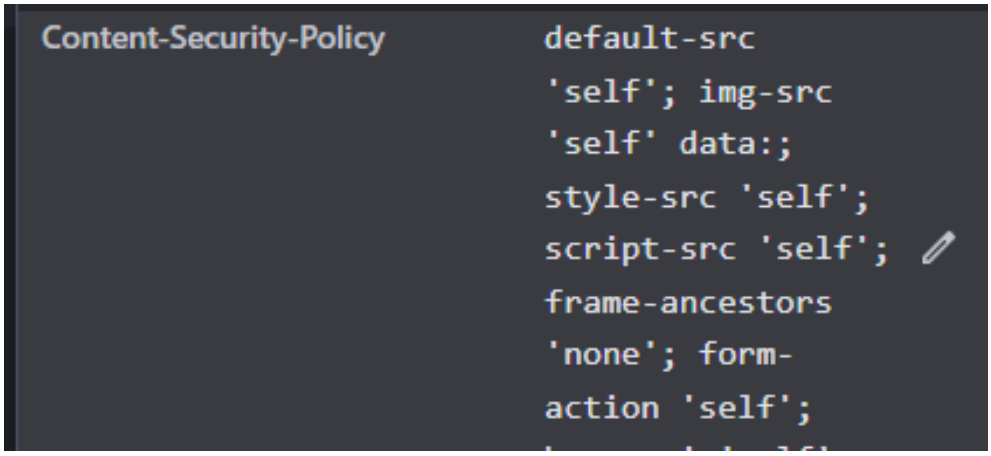
Objetivo: Comprobar el bloqueo temporal sin revelar si el correo existe.
Resultado observado: Tras cinco intentos se bloquea el acceso con ese correo durante 5 minutos, con cuenta regresiva; incluso la contraseña correcta es rechazada. Otras cuentas pueden iniciar sesión.
Control aplicado: Contador de intentos y bloqueo por usuario en la base de datos y por sesión, con el mismo mensaje para correos existentes e inexistentes.



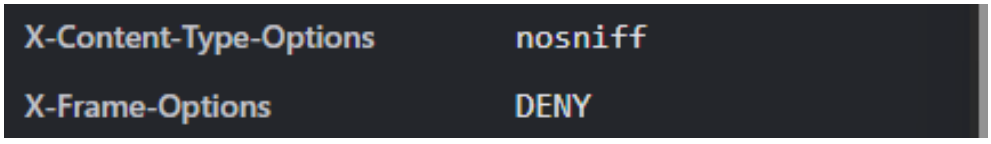
Bloqueo temporal con cuenta regresiva

Prueba 7 – Inspección de cabeceras HTTP

Objetivo: Verificar que el servidor envía las cabeceras de seguridad configuradas.
Resultado observado: Se observan Content-Security-Policy, X-Content-Type-Options: nosniff y X-Frame-Options: DENY.
Control aplicado: Cabeceras enviadas desde includes/bootstrap.php en todas las páginas.



Cabecera Content-Security-Policy



Cabeceras X-Content-Type-Options y X-Frame-Options

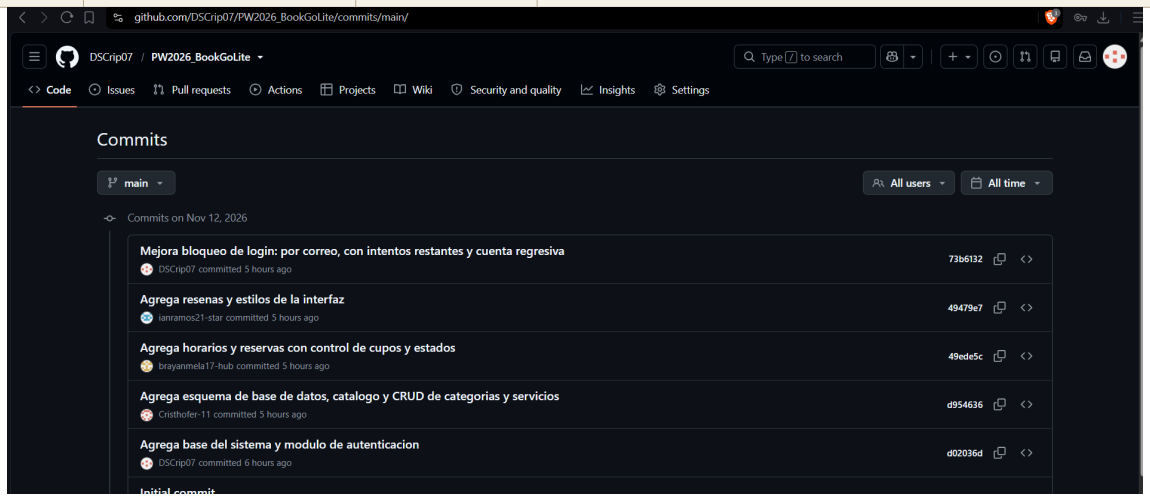
7. Distribución del trabajo y GitHub

Repositorio compartido: **https://github.com/DSCrip07/PW2026_BookGoLite**. Los cuatro integrantes fueron incorporados como colaboradores y subieron su módulo con commits desde su propia cuenta.

Módulo	Responsable	Archivos principales
Autenticación y sesiones	Josua Martinez	includes/bootstrap.php, header.php, footer.php, password_policy.php; public/login.php, registro.php, logout.php; database/create_admin.php
Catálogo y administración	Cristhofer Villarreal	database/schema.sql; public/index.php, catalogo.php, servicio.php; public/admin/index.php, categorias.php, servicios.php
Horarios y reservas	Brayan Mela	includes/reservas.php; public/reservar.php, mis_reservas.php; public/admin/horarios.php, reservas.php
Reseñas, interfaz y pruebas	Ian Ramos	public/resenar.php; public/css/style.css; security/SECURITY.md y evidencias

Historial de contribuciones

Commit	Autor	Fecha	Descripción
70beea8	DSCrip07	2026-11-12	Initial commit
d02036d	Josua Martinez	2026-11-12	Agrega base del sistema y modulo de autenticacion
d954636	Cristhofer Villarreal	2026-11-12	Agrega esquema de base de datos, catalogo y CRUD de categorias y servicios
49ede5c	Brayan Mela	2026-11-12	Agrega horarios y reservas con control de cupos y estados
49479e7	Ian Ramos	2026-11-12	Agrega resenas y estilos de la interfaz
73b6132	Josua Martinez	2026-11-12	Mejora bloqueo de login: por correo, con intentos restantes y cuenta regresiva



Historial de commits en GitHub

8. Uso de inteligencia artificial

En cumplimiento de la sección 11 del enunciado, el grupo declara el uso de herramientas de IA:

Herramienta: Claude Code (Anthropic), asistente de IA usado desde la aplicación de escritorio de Claude.

Para qué se usó:

- Generación del código base de la aplicación (PHP, SQL, CSS y JavaScript) a partir del enunciado.
- Diseño del esquema de siete tablas y de las restricciones de integridad.
- Implementación de los controles de seguridad (contraseñas, sesiones, CSRF, XSS, SQL Injection, bloqueo de intentos y cabeceras).
- Implementación de la política de contraseñas con carácter especial y del bloqueo por intentos, solicitados por el profesor.
- Diagnóstico y corrección de errores encontrados al probar: acentos corruptos al importar el SQL, hoja de estilos que no cargaba en XAMPP por la ruta base, desajuste de zona horaria en el bloqueo y código HTTP 419 que Apache convertía en 500.
- Redacción de security/SECURITY.md, del diagrama entidad-relación, de este informe y de los borradores de las reflexiones individuales, que cada integrante debe revisar y ajustar.

Verificación: la IA ejecutó pruebas por consola de las reglas de negocio (cupos, estados, duplicados, propiedad de recursos) y de los controles de seguridad; el grupo reprodujo las pruebas de seguridad en el navegador y tomó las capturas que se incluyen como evidencia.

Aportes del grupo: elección del negocio, revisión y prueba de la aplicación en su propio entorno (XAMPP), creación del repositorio y de los commits desde las cuentas de cada integrante, captura de evidencias y revisión de las reflexiones individuales.

Cada integrante se compromete a poder explicar el código de su módulo y las consultas, relaciones, permisos y validaciones que el profesor solicite durante la sustentación.

9. Reflexión individual

Josua Martinez – Autenticación y sesiones

Mi módulo fue la autenticación y las sesiones: registro, inicio y cierre de sesión, la política de contraseñas y el bloqueo por intentos fallidos. Lo más importante que entendí es que ocultar un botón o un menú no es seguridad: cada página valida el rol en PHP, porque cualquiera puede escribir la dirección directamente. Entendí también por qué las contraseñas se guardan con password_hash y se comparan con password_verify, y por qué el mensaje de error es el mismo exista o no el correo. Implementamos la contraseña con carácter especial obligatorio y el bloqueo tras cinco intentos, que se aplica por correo para que otra cuenta pueda seguir entrando. Además ejecuté las pruebas de seguridad en el navegador y tomé las capturas. Puedo explicar login.php, registro.php, password_policy.php y cómo funcionan el token CSRF y la regeneración de la sesión. Parte del código se desarrolló con asistencia de IA; lo revisé y lo probé en mi equipo.

Cristhofer Villarreal – Catálogo y administración

Mi módulo fue el esquema de la base de datos, el catálogo y la administración de categorías y servicios. Aprendí cómo las claves foráneas y las restricciones protegen los datos desde la base: el precio no puede ser negativo, la duración debe ser mayor que cero y una categoría o servicio con dependencias no se elimina físicamente, se desactiva. En el catálogo, la búsqueda por nombre y el filtro por categoría usan consultas preparadas, por lo que una entrada como ' OR '1'='1 no altera la consulta. Puedo explicar las relaciones entre las siete tablas, el archivo schema.sql, catalogo.php y los formularios del CRUD en public/admin. Parte del código se desarrolló con asistencia de IA; lo revisé y lo probé en mi equipo.

Brayan Mela – Horarios y reservas

Mi módulo fue el de horarios y reservas, que es la parte con más reglas de negocio. Lo central que entendí es que el control de cupos no puede depender de deshabilitar un botón: la comprobación y el registro de la reserva se hacen juntos en una transacción, con un UPDATE condicionado que solo descuenta si quedan cupos. También entendí por qué cancelar dos veces no devuelve dos cupos: el cambio de estado solo ocurre si la reserva sigue PENDIENTE o CONFIRMADA. La clave foránea compuesta evita reservar un horario que no pertenece al servicio. Puedo explicar includes/reservas.php, el flujo PENDIENTE, CONFIRMADA y REALIZADA, y las validaciones de horarios (no en el pasado y hora final posterior a la inicial). Parte del código se desarrolló con asistencia de IA; lo revisé y lo probé en mi equipo.

Ian Ramos – Reseñas, interfaz y pruebas de seguridad

Mi módulo fue el de reseñas, la interfaz y las pruebas. Una reseña solo se puede publicar sobre una reserva propia en estado REALIZADA y una sola vez por reserva, y esa regla se valida en el servidor y también con una restricción única en la base de datos. Los comentarios se muestran con htmlspecialchars, por eso un texto como <script> aparece como texto y no se ejecuta; lo comprobamos en una prueba de XSS. En la interfaz usamos CSS propio con tema carbón y dorado, adaptable a teléfono. Puedo explicar resenar.php, cómo se muestra el promedio de calificaciones en el detalle del servicio y cada una de las siete pruebas de seguridad de security/SECURITY.md. Parte del código se desarrolló con asistencia de IA; lo revisé y lo probé en mi equipo.